#decoded

FastAPI

Why AI Backend Choose It

Build powerful, scalable and production-ready APIs with Python.

- High Performance
- Automatic Docs
- Data Validation
- Easy Deployment

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def home():
    return {"message":
            "Hello, World!"}
```



@darpan.decoded

01 FOUNDATION

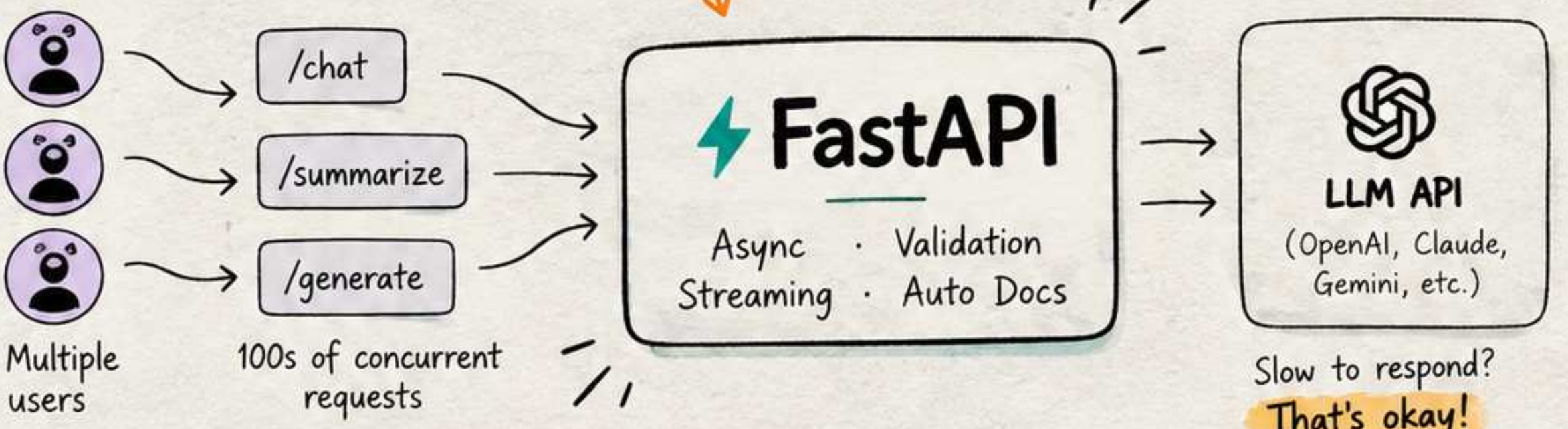
#decoded01

Why FastAPI Became the AI Backend Standard

Better APIs
Better AI Produ :)

Built for AI workloads (async + validated + production ready)

Your LLM isn't slow.
Your backend usually is.



Async
One worker handles many waiting model calls.

Pydantic
Validates requests and responses automatically.

Streaming
Send tokens as they arrive.

Auto Docs
OpenAPI docs generated from your types.

```python
# main.py
from fastapi import FastAPI

app = FastAPI(title="AI Service")

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Even a simple route is async 👀

Decoded Tip: FastAPI shines because AI backends spend most of their time waiting on external APIs – not burning CPU.



@darpan.decoded

Build Learn Share

#decoded02

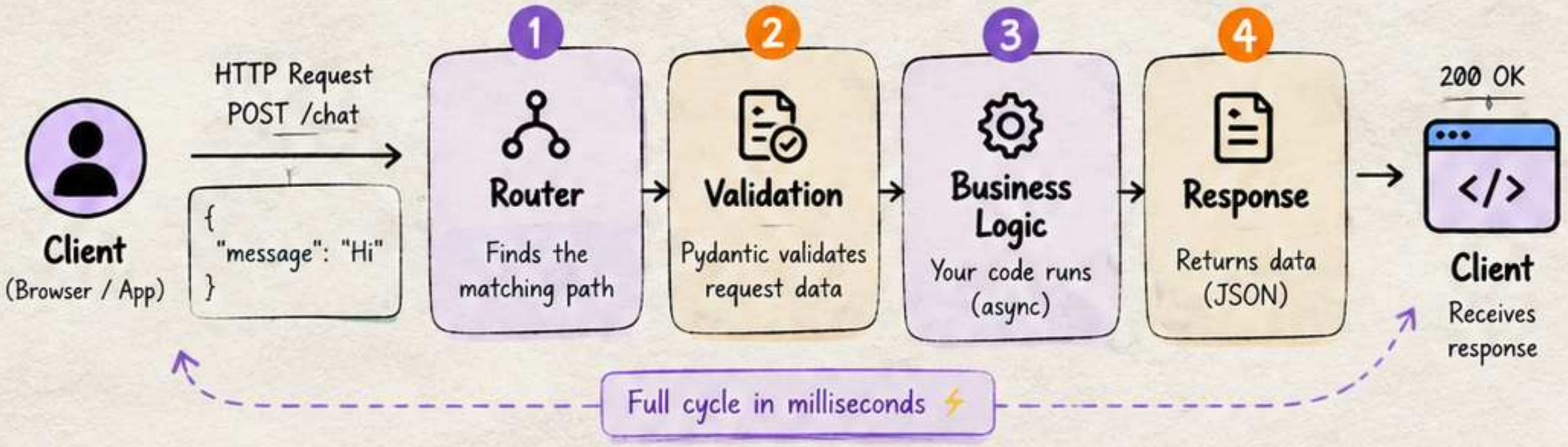
The Request Lifecycle

Same Request, Multiple Checks Behind the Scenes

What happens behind a single FastAPI request?

A lot more than just calling a function. Here's the journey from the moment a request hits your API to the response that reaches the user.

Same request, multiple layers of magic ✨



main.py

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
async def chat(req: ChatRequest):
    # your AI logic here
    return {"reply": "Hello!"}
```

FastAPI handles routing, validation and async execution for you.

Step by Step

1 Routing
FastAPI matches the incoming request with the correct path operation function.

/chat → chat()

2 Validation
Pydantic validates and parses the request body, query params, headers, etc.

Invalid data? Returns 422

3 Business Logic
Your endpoint function runs (usually async) – call LLMs, DBs, or other services here.

Use async to avoid blocking

4 Response
FastAPI serializes the output (dict / model) into JSON and sends it back.

Automatic JSON serialization

Decoded Tip: Keep your endpoints thin. Let FastAPI handle the HTTP stuff, and focus on your actual logic (AI, DB, business rules).

Clean APIs Scale Faster :)

@darpan.decoded



#decoded03

A Project Structure That Doesn't Become a Mess

Clean Structure Happy Developers :)

The biggest FastAPI mistake is putting everything in main.py. A clean structure keeps your project scalable and maintainable.

Good structure today = faster features tomorrow.

```
Project structure
app/
    main.py              # create app, include routers
    config.py            # load environment variables
    routers/
        chat.py          # API route handlers
        users.py
    schemas/
        chat.py          # request/response models
        user.py
    services/
        llm.py           # business logic (AI, DB, etc.)
        vector_store.py
    db/
        session.py       # database connection
    core/
        settings.py      # app settings, auth, utils
        auth.py
```

One responsibility per folder.

main.py
Creates the FastAPI app and includes all routers.
→ Entry point of your app

routers/
Defines API endpoints (HTTP layer only).
→ Keep it thin. No heavy logic here.

schemas/
Pydantic models for request and response validation.
→ Validates data automatically ✨

services/
Core business logic (LLM calls, embeddings, etc.).
→ Your AI logic lives here.

db/
Database connection and session management.
→ Reuse connections, don't recreate in every route.

core/
Settings, environment variables, auth utilities, helper functions.
→ Keep app configuration separate.



Decoded Tip: Separate your concerns early. Routers handle HTTP, services handle intelligence, and your app stays scalable.

Build Learn Share :)

#decoded04

Pydantic Validation

Your Invisible Security Guard

Validate
Early
Fail Fast
Build Better
:)

Clean data
Happy code
Fewer bugs ☺

Never trust incoming data. Pydantic (built into FastAPI) automatically validates, parses and documents your request & response data.

Invalid Request (No Validation)

```
{
   "message": 123,
   "temperature": "hot",
   "max_tokens": -10
}
```

should be string
should be a number
cannot be negative

Without validation, this reaches your logic and can break things.

Pydantic catches this before it reaches your code!

Valid Request (With Pydantic)

```
{
   "message": "Explain FastAPI",
   "temperature": 0.7,
   "max_tokens": 200
}
```

Correct types
Clean data
Ready for use

Only valid, well-structured data reaches your endpoint function.

schemas/chat.py

```python
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    message: str = Field(...,  min_length=1,
                         max_length=1000,
                         description="User message")
    temperature: float = Field(0.7, ge=0.0, le=2.0)
    max_tokens: int = Field(200, gt=0, le=4000)

class ChatResponse(BaseModel):
    reply: str
    usage_tokens: int
```

Define the shape of your data using Python types.

main.py

```python
from fastapi import FastAPI
from schemas.chat import ChatRequest, ChatResponse

app = FastAPI()

@app.post("/chat", response_model=ChatResponse)
async def chat(req: ChatRequest):
    # your AI logic here
    return ChatResponse(
        reply="Hello!",
        usage_tokens=42
    )
```

FastAPI uses your model for validation, parsing and automatic docs.

Why It's Powerful?

Validation
Rejects invalid data automatically.

Parsing
Converts types for you (e.g. str → int).

Documentation
Used to generate OpenAPI docs.

Type Safety
Catches errors early before they hit your logic.

Decoded Tip:
Use Pydantic models for both request and response.
It makes your API safer, self-documenting and easier to maintain.

Types Save Time :)

Better APIs Happier Developers :)

@darpan.decoded

#decoded05

Async vs Sync

Same server. 100 users. Huge difference.

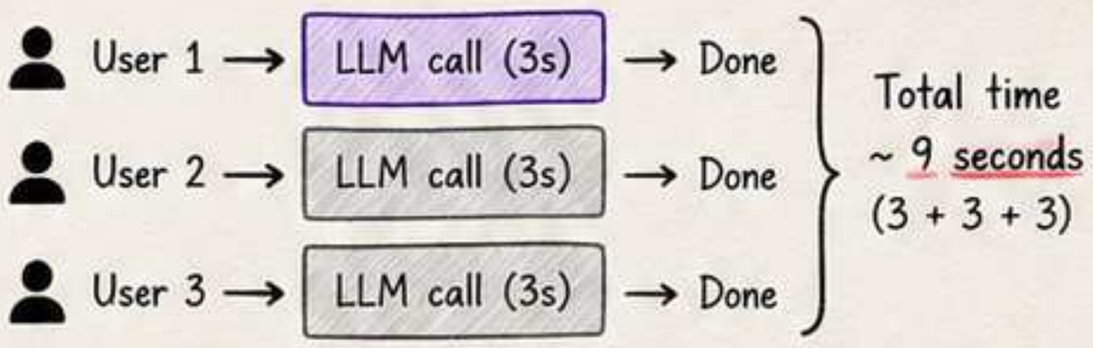
AI apps mostly wait — for LLM responses, database queries, or external APIs. Async helps you handle many **waiting requests** without blocking each one.

Async doesn't make your code run faster, it helps you do more while waiting.

More Requests Happier Users :)

Synchronous (blocking)

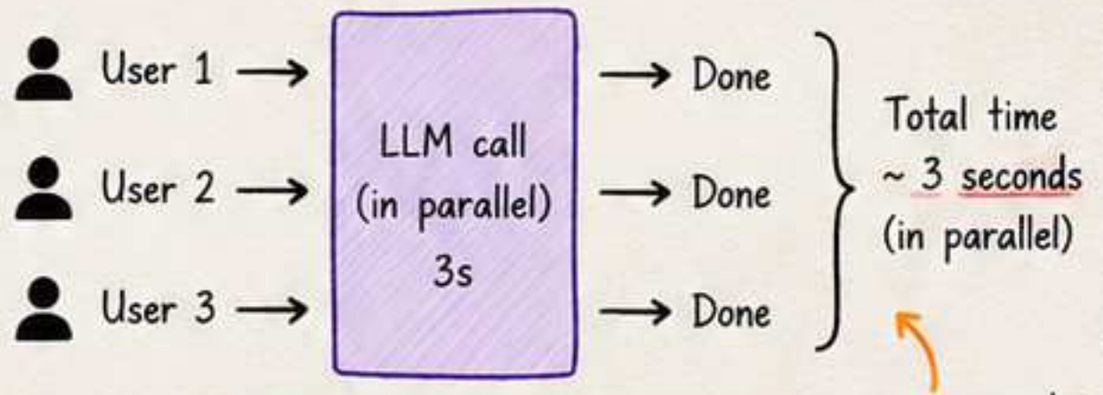
Each request waits for the previous one to finish. Others are blocked.



Server sits idle while waiting for each response.

Asynchronous (non-blocking)

All requests are sent, and the server waits for all of them at the same time.



One worker can handle many waiting requests.

This is why FastAPI is great for AI workloads

```python
# sync_example.py
import time
from fastapi import FastAPI

app = FastAPI()

@app.get("/sync")
def sync_endpoint():
    time.sleep(3)   # blocks the worker
    return {"message": "done"}
```

Blocks the server for 3 seconds.

```python
# async_example.py
import asyncio
from fastapi import FastAPI

app = FastAPI()

@app.get("/async")
async def async_endpoint():
    await asyncio.sleep(3)   # non-blocking
    return {"message": "done"}
```

Lets the server handle other requests while waiting.

When to use what?

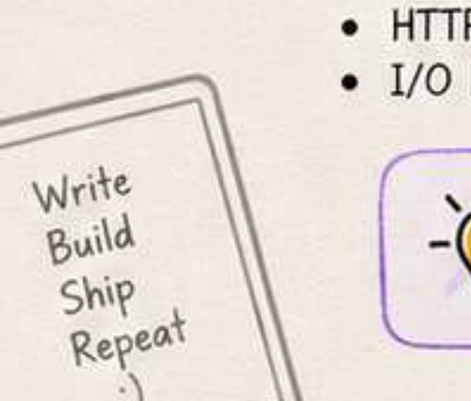
Use async for:

- Calling LLM APIs (OpenAI, Claude, etc.)
- Database queries (async drivers)
- HTTP requests to external services
- I/O bound tasks (files, network, APIs)

Use sync for:

- CPU heavy work (image processing, training)
- Simple scripts / one-off tasks
- When the library doesn't support async

Async speeds up waiting, not computation.



Decoded Tip: In AI applications, most of your time is spent waiting on external APIs. Async helps you make the most of that time.

Faster Backends Happier Developers :)

#decoded06

Streaming

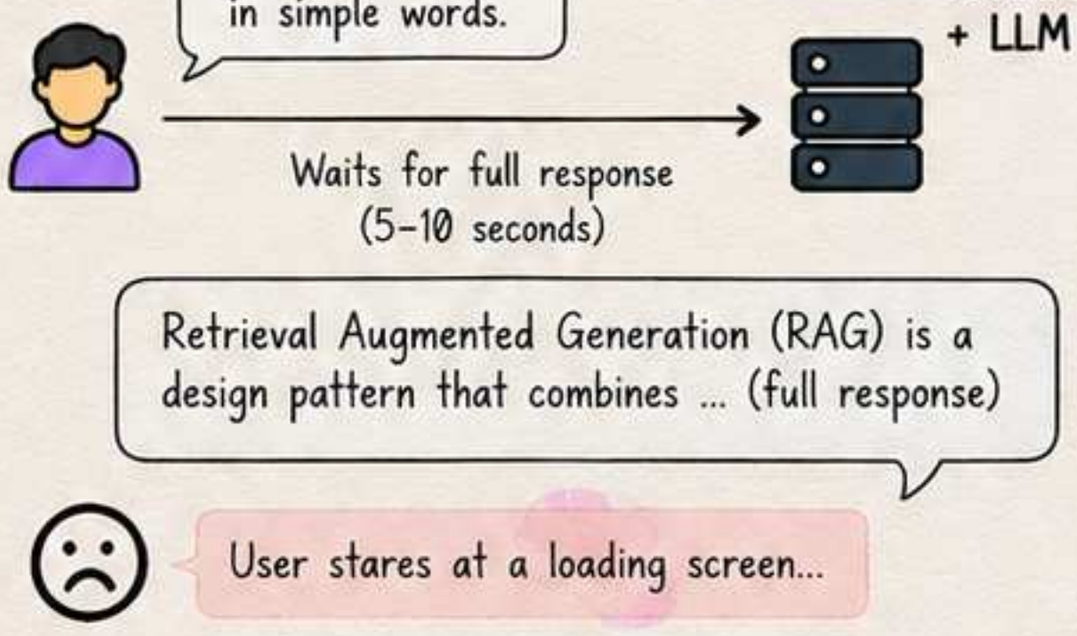
LLM Responses with FastAPI

Same response.
Different experience.
Streaming feels instant ⚡

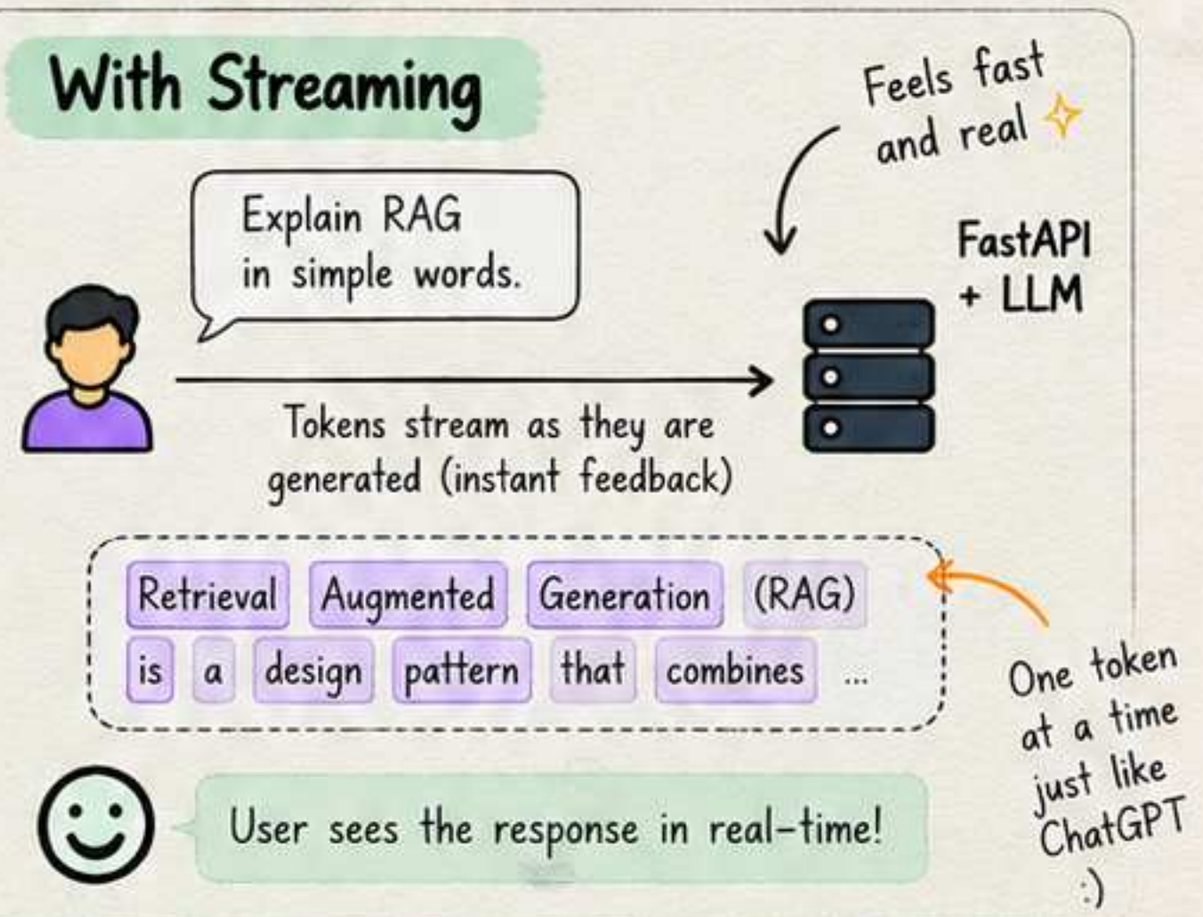
Faster Apps
Happier Users
:)

Give your users a ChatGPT-like experience — tokens appear as they are generated, instead of waiting for the full response.

Without Streaming



With Streaming



main.py

```python
from fastapi import FastAPI, StreamingResponse
from openai import AsyncOpenAI

app = FastAPI()
client = AsyncOpenAI()

@app.get("/stream")
async def stream():
    async def event_generator():
        response = await client.chat.completions.create(
            model="gpt-4o",
            messages=[{"role": "user", "content": "Explain RAG in simple words."}],
            stream=True,
        )
        async for chunk in response:
            token = chunk.choices[0].delta.content
            if token:
                yield token   # send each token as soon as it arrives

    return StreamingResponse(event_generator(), media_type="text/plain")
```

Send each token as it arrives

How It Works?

1. Client makes a request (with stream=True)
2. LLM generates tokens one by one
3. We yield each token as it arrives
4. User receives tokens in real-time (same as ChatGPT)

Use Cases

- Chatbots / AI assistants
- Long-form content generation
- Real-time agents
- Better user experience (lower perceived latency)

Decoded Tip: Streaming doesn't make the model faster, but it makes your app **feel 10x faster.**

Streaming = Better Products :)

Build Learn Share :)



@darpan.decoded

#decoded07

Dependency Injection

FastAPI's Secret Weapon

Stop creating database connections, auth clients or configs inside every endpoint. Use **Depends()** and let FastAPI handle it.

Same logic.
Reusable.
Cleaner code.
Easier testing.

Write Less Scale More :)

Without Dependency Injection

```python
@app.get("/users")
async def get_users():
    db = create_db_session()
    user = get_current_user()
    # business logic
    return {"users": [...]}
```

- Creates a new DB session every time
- Repeated auth logic
- Harder to test and maintain

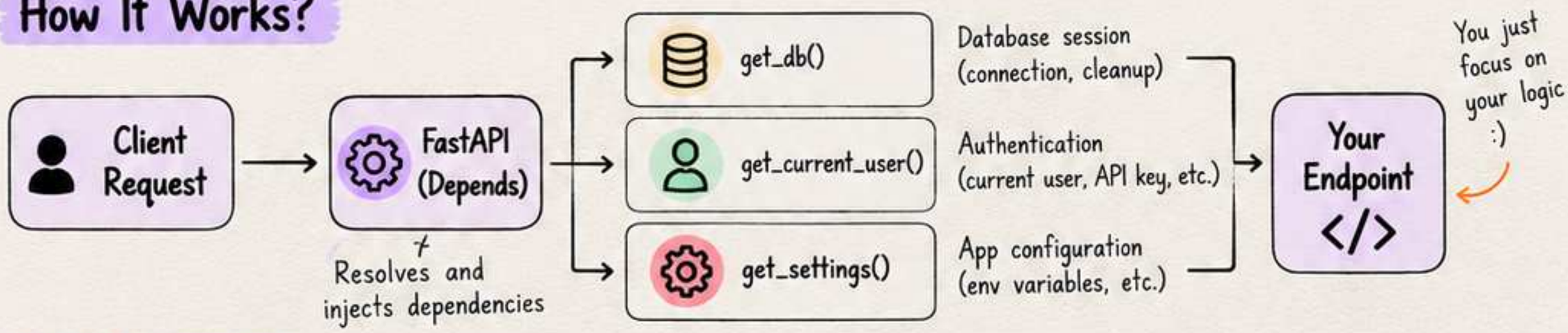
With Dependency Injection

```python
from fastapi import Depends

@app.get("/users")
async def get_users(
    db: Session = Depends(get_db),
    user: User = Depends(get_current_user)
):
    # business logic
    return {"users": [...]}
```

- Reuses the same logic across endpoints
- Clean, readable and testable
- FastAPI handles the lifecycle

How It Works?



Real Examples

database.py

```python
from sqlalehemp import Session

def get_db():
    db =SessionLocal()
    try:
     ↗ yield db
    finally:
        db.close()
```

Manage DB session automatically (even cleanup)

auth.py

```python
from fastapi import HTTPException

def get_current_user():
    token = ...  # verify token
    if not token:
        raise HTTPException(
            status_code=401,
            detail="Invalid token"
    )
    return user
```

Reuse authentication across multiple routes

config.py

```python
from pydantic_settings import
    BaseSettings

class Settings(BaseSettings):
    app_name: str = "AI API"
    debug: bool = False

@lru_cache()
def get_settings():
    return Settings()
```

Inject app settings (environment variables)

Modular Scalable Maintainable :)

Decoded Tip: Dependencies make your code cleaner, reusable, and ridiculously easy to test.

Better Backends Happier Developers :)

@darpan.decoded

#decoded08

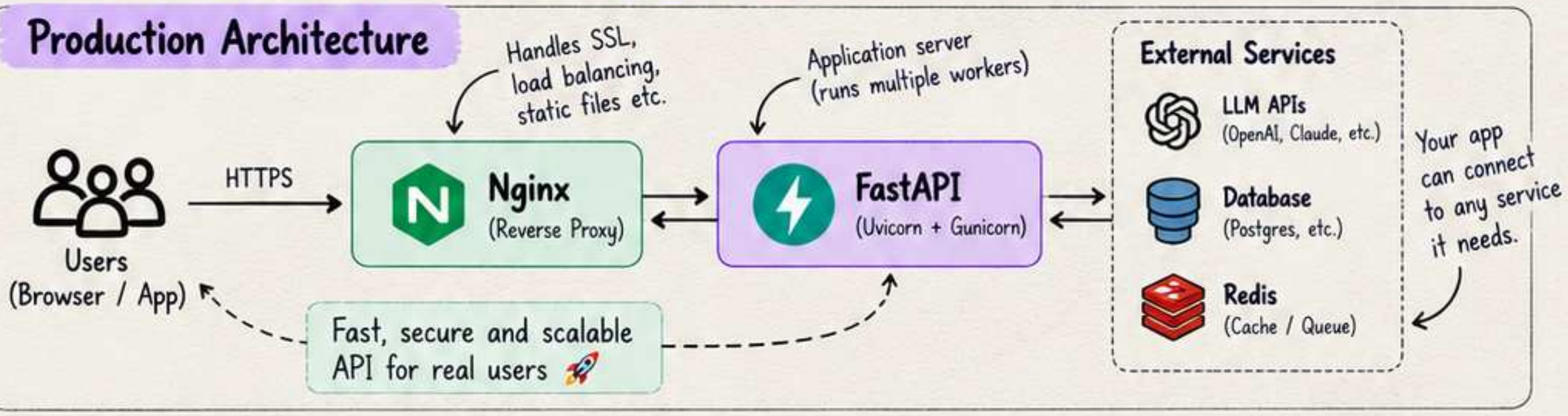
Deploying FastAPI
From Localhost to Production

Same app.
Bigger impact.
Deploy it right for real users. 🚀

Build
Deploy
Scale
Repeat
:)

Running a FastAPI app locally is easy. Deploying it the right way makes it **reliable, secure** and ready for **real users.**

Production Architecture



Key Deployment Steps

1. **Prepare your app**
   Clean structure, environment variables, production settings.
2. **Use a production server**
   Run with Uvicorn + Gunicorn (multiple workers).
3. **Add a reverse proxy**
   Use Nginx (or Cloud Load Balancer) for HTTPS, routing and security.
4. **Containerize with Docker**
   Keep your environment consistent.
5. **Set up monitoring & logs**
   Track errors, latency and usage.
6. **Keep /docs private**
   Disable in production or protect with auth.

Small steps
Big impact
:)

Run with Uvicorn + Gunicorn

terminal

```
# install dependencies
pip install fastapi uvicorn gunicorn

# run with multiple workers (production)
gunicorn app.main:app \
  -k uvicorn.workers.UvicornWorker \
  --workers 4 --bind 0.0.0.0:8000
```

Uvicorn = ASGI server
Gunicorn = process manager

Multiple workers handle more traffic 🚀

Dockerize

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["gunicorn", "app.main:app", "-k",
  "uvicorn.workers.UvicornWorker", "--workers",
   "4", "--bind", "0.0.0.0:8000"]
```

Same setup everywhere.
No "works on my machine" 😅

Environment Variables
Use .env for secrets (API keys, DB URLs, etc.).

Monitoring
Add logging, error tracking and alerts (e.g. Sentry).

Secure your App
Use HTTPS, rate limiting and authentication.

Disable Docs (Prod)
Set `docs_url=None` or protect with auth.



Decoded Tip: A clean deployment setup saves you from 90% of production issues. Invest in it early.

Better Tools Happier Developers :)

Learn
Build
Deploy
Repeat

#decoded

Get the Full FastAPI Guide

Step-by-step code, real examples,
deployment tips and best practices.

Comment

FASTAPI

for full PDF guide in DM.

A
Better
Developer
Today

Good
Code
Better
Products

Clean Code
Designing Data-Intensive Applications
System Design Interview
The Pragmatic Programmer

Build
Good
Things

Same
Python.
Bigger
Possibilities.

Ideas → Products
Build
Ship
Repeat
:)

@darpan.decoded

Keep
Building
:)